Python for Artificial Intelligence

Pandas I: DataFrames & Selecting Data

Week 3, Day 2

Today's Agenda

- Hour 1: Reading a CSV, DataFrame and Series basics, .head(), .info(), .describe()
- Hour 2: Selecting columns and rows, filtering with conditions, sorting
- Hour 3: Handling missing values, and practice combining everything on one dataset
- By the end: you can load a real dataset and answer basic questions about it in a few lines

Hour 1

DataFrames, Series & First Look

CONCEPT

Why Do We Need Pandas?

- A NumPy array is a grid of numbers, but real data has named columns, mixed types, and missing values
- Pandas (Panel Data) adds labels on top of NumPy: named columns, a row index, and tools built for messy real-world data
- Pandas is how you load a spreadsheet or CSV file into Python and start asking it questions
- Under the hood, each Pandas column is stored as a NumPy array

CONCEPT

Series vs. DataFrame

- A Series is a single labeled column: values plus an index
- A DataFrame is a full table: multiple Series sharing the same row index
- Selecting one column from a DataFrame (df['score']) gives you back a Series
- Think of a DataFrame as a dictionary of columns, each one a Series

Reading Data & Creating a DataFrame

SYNTAX

```
import pandas as pd

df = pd.read_csv("students.csv")
```

PARAMETERS

pd the conventional short name for the pandas module after import

"students.csv" the path to a CSV file; can also be built from a dict with pd.DataFrame(dict)

YOU CAN

- ✓ Load a real CSV file into a DataFrame in one line
- ✓ Build a small DataFrame directly from a Python dictionary for testing
- ✓ Immediately inspect it with .head(), .info(), or .describe()

YOU CANNOT

- ✗ Read a file that doesn't exist at that path without an error
- ✗ Assume column names or order without checking .columns first

Returns: a new DataFrame object

EXAMPLE

Building a DataFrame from a Dictionary

Each key becomes a column name; each list becomes that column's values.

```
import pandas as pd

data = {
    "name": ["Ali", "Sara", "Bilal"],
    "city": ["Karachi", "Lahore", "Karachi"],
    "score": [85, 92, 78],
}

df = pd.DataFrame(data)
print(df)
# a 3-row table with columns: name, city, score
```

THINK & PRACTICE

Quick Check

Build a DataFrame named products from a dictionary with columns "item" (values: "Pen", "Notebook", "Bag") and "price" (values: 20, 60, 900). Print it.

ANSWER

Quick Check: Answer

```
import pandas as pd

data = {
    "item": ["Pen", "Notebook", "Bag"],
    "price": [20, 60, 900],
}

products = pd.DataFrame(data)
print(products)
```

CONCEPT

Getting Your First Look at a DataFrame

- df.head(n) shows the first n rows (default 5); df.tail(n) shows the last n
- df.info() lists every column, its data type, and how many non-missing values it has
- df.describe() gives count, mean, min, max, and quartiles for every numeric column
- df.shape gives (rows, columns); df.columns lists the column names

EXAMPLE

Inspecting a Students DataFrame

These four calls are usually the first thing you run on any new dataset.

```
import pandas as pd

df = pd.DataFrame({
    "name": ["Ali", "Sara", "Bilal", "Hina"],
    "age": [22, 21, 23, 20],
    "score": [85, 92, 78, 88],
})

print(df.shape)     # (4, 3)
print(df.columns)   # Index(['name', 'age', 'score'])
print(df.head(2))   # first 2 rows
```

THINK & PRACTICE

Quick Check

Using the products DataFrame from before, print df.shape and df.describe(). Which column does .describe() actually summarize, and why?

ANSWER

Quick Check: Answer

```
print(products.shape)      # (3, 2)
print(products.describe())

# describe() only summarizes the "price" column,
# because it is numeric; "item" is text (an object
# column), so there is no mean or quartile to compute
```

THINK & PRACTICE

Think & Practice (Harder)

Build a DataFrame orders with columns "order_id" ([1,2,3,4,5]), "amount" ([500, 1200, 300, 900, 150]). Print df.info() and, separately, the single highest amount using df['amount'].max(). Explain in a comment what df.info() tells you that df.describe() does not.

ANSWER

Think & Practice (Harder): Answer

```
import pandas as pd

orders = pd.DataFrame({
    "order_id": [1, 2, 3, 4, 5],
    "amount": [500, 1200, 300, 900, 150],
})

print(orders.info())
print(orders["amount"].max())  # 1200

# info() shows data types and missing-value counts;
# describe() shows statistics, but not types or gaps
```

Hour 2

Selecting, Filtering & Sorting

CONCEPT

Selecting Columns and Rows

- df['col'] selects one column as a Series; df[['col1', 'col2']] selects several columns as a DataFrame
- df.loc[row_label, col_label] selects by label; df.iloc[row_position, col_position] selects by position
- df.loc[2] gives row with index label 2; df.iloc[0] always gives the first row, regardless of labels
- A single square bracket with a list of column names is the most common way to grab a subset of columns

EXAMPLE

Selecting Columns and Rows

Square brackets pick columns; .loc and .iloc pick rows (and optionally columns too).

```
import pandas as pd

df = pd.DataFrame({
    "name": ["Ali", "Sara", "Bilal"],
    "city": ["Karachi", "Lahore", "Karachi"],
    "score": [85, 92, 78],
})

print(df["name"])          # Series of names
print(df[["name", "score"]])  # DataFrame, 2 columns
print(df.loc[1])           # Sara's full row, by label
print(df.iloc[0])          # Ali's full row, by position
```

THINK & PRACTICE

Quick Check

Using the students DataFrame ({name, age, score} for Ali, Sara, Bilal, Hina), select and print just the "age" column, then select and print the row at position 2 using .iloc.

ANSWER

Quick Check: Answer

```
print(df["age"])   # Series of ages
print(df.iloc[2])  # Bilal's full row
```

CONCEPT

Filtering Rows with Conditions

- df['score'] > 80 creates a Series of True/False values, one per row
- Passing that Series back into df[...] keeps only the rows where it's True, this is a boolean mask
- Combine conditions with & (and) and | (or); wrap each condition in its own parentheses
- This is the Pandas equivalent of a filter() call, but reads much more directly

EXAMPLE

Filtering with a Boolean Mask

The comparison builds a mask; the mask selects only the matching rows.

```
import pandas as pd

df = pd.DataFrame({
    "name": ["Ali", "Sara", "Bilal", "Hina"],
    "city": ["Karachi", "Lahore", "Karachi", "Islamabad"],
    "score": [85, 92, 78, 88],
})

high_scorers = df[df["score"] > 80]
print(high_scorers)  # Ali, Sara, Hina

karachi_high = df[(df["city"] == "Karachi") & (df["score"] > 80)]
print(karachi_high)  # Ali only
```

THINK & PRACTICE

Quick Check

Using the orders DataFrame ({order_id, amount} for 5 orders), filter and print only the orders where amount is greater than or equal to 500.

ANSWER

Quick Check: Answer

```
big_orders = orders[orders["amount"] >= 500]
print(big_orders)  # order_id 1, 2, 4
```

CONCEPT

Sorting a DataFrame

- df.sort_values('col') sorts rows by that column, smallest to largest by default
- Pass ascending=False to sort largest to first
- Sort by more than one column with a list: df.sort_values(['col1', 'col2'])
- Sorting returns a new DataFrame; it doesn't change the original unless you reassign it

EXAMPLE

Sorting Students by Score

ascending=False gives the highest score first, useful for a leaderboard.

```
sorted_df = df.sort_values("score", ascending=False)
print(sorted_df)
# Sara (92), Hina (88), Ali (85), Bilal (78)
```

THINK & PRACTICE

Think & Practice (Harder)

Using the orders DataFrame, filter to keep only orders with amount over 200, then sort the result by amount in descending order. Do this in two steps (filter, then sort), and print the final result.

ANSWER

Think & Practice (Harder): Answer

```
filtered = orders[orders["amount"] > 200]
result = filtered.sort_values("amount", ascending=False)
print(result)
# amount: 1200, 900, 500, 300 with their order_ids
```

15-Minute Break

Stretch, grab a coffee. We'll resume shortly.

Hour 3

Missing Values & Practice

CONCEPT

Handling Missing Values

- Real data almost always has gaps; Pandas marks a missing value as NaN (Not a Number)
- df.isna() returns True/False for every cell, showing exactly where data is missing
- df.dropna() removes any row that has at least one missing value
- df.fillna(value) replaces every missing value with something specific, like 0 or a column average

EXAMPLE

Finding and Handling Gaps in Data

Bilal's score is missing (NaN); each method handles it differently.

```
import pandas as pd
import numpy as np

df = pd.DataFrame({
    "name": ["Ali", "Sara", "Bilal"],
    "score": [85, 92, np.nan],
})

print(df.isna())        # True only in Bilal's score row
print(df.dropna())      # keeps only Ali and Sara
print(df.fillna(0))     # Bilal's score becomes 0
```

THINK & PRACTICE

Quick Check

Given a DataFrame with a "city" column where one value is missing (np.nan), fill every missing value in "city" with the string "Unknown" using .fillna(), and print the result.

ANSWER

Quick Check: Answer

```
import pandas as pd
import numpy as np

df = pd.DataFrame({"city": ["Karachi", np.nan, "Lahore"]})

filled = df.fillna("Unknown")
print(filled)
# Karachi, Unknown, Lahore
```

WORKED EXAMPLE

Putting It Together: Clean, Filter, Sort

A realistic mini-pipeline: fill missing scores, then filter and sort the cleaned data.

```
import pandas as pd
import numpy as np

df = pd.DataFrame({
    "name": ["Ali", "Sara", "Bilal", "Hina"],
    "score": [85, 92, np.nan, 88],
})

clean = df.fillna(df["score"].mean())
passing = clean[clean["score"] >= 85]
result = passing.sort_values("score", ascending=False)

print(result)
```

THINK & PRACTICE

Practice

Using a DataFrame with columns {name, attendance} for 5 students where one attendance value is missing (np.nan), drop the row with the missing value using .dropna(), then sort the remaining students by attendance descending. Print the result.

ANSWER

Practice: Answer

```
import pandas as pd
import numpy as np

df = pd.DataFrame({
    "name": ["Ali", "Sara", "Bilal", "Hina", "Omar"],
    "attendance": [18, 20, np.nan, 15, 19],
})

clean = df.dropna()
result = clean.sort_values("attendance", ascending=False)
print(result)
```

THINK & PRACTICE

Practice (Harder)

Using a DataFrame of 6 products with columns {name, price, in_stock} where in_stock is True/False and one price is missing (np.nan): fill the missing price with the average of the other prices, then filter to keep only products where in_stock is True, then sort by price ascending. Print the final result.

ANSWER

Practice (Harder): Answer

```
import pandas as pd
import numpy as np

df = pd.DataFrame({
    "name": ["Pen", "Bag", "Notebook", "Desk", "Lamp", "Chair"],
    "price": [20, 900, 60, np.nan, 450, 1500],
    "in_stock": [True, True, False, True, True, False],
})

filled = df.fillna(df["price"].mean())
available = filled[filled["in_stock"]]
result = available.sort_values("price")

print(result)
```

Home Practice: Interview-Style Questions

- What is the difference between a Pandas Series and a DataFrame?
- What is the difference between .loc[] and .iloc[]?
- How does a boolean mask work when filtering a DataFrame?
- What is the difference between .dropna() and .fillna(), and when would you choose each?
- Why does df.describe() sometimes skip a column entirely?
- Given a DataFrame of sales, write one line that returns only rows above the average sale amount

Key Takeaways & Coming Up

- A DataFrame is a table of labeled columns (Series); read one from disk with pd.read_csv()
- .head(), .info(), and .describe() are your first three moves on any new dataset
- Square brackets and .loc / .iloc select columns and rows; a boolean mask filters rows by condition
- .isna(), .dropna(), and .fillna() are the standard toolkit for missing data
- Coming up next: Pandas II, grouping and aggregating data, merging DataFrames, and exporting your results